You're almost there — sign up to start building in Notion today. Sign up or login

week-03-css-fundamentals

Week 03 — CSS Fundamentals

Session goal: by the end of class you have a styled card (and a restyled profile) driven by an external stylesheet — box model, cascade, and custom properties under your fingers.

CSS describes how HTML looks. You write rules: a selector picks elements, then properties set colours, sizes, spacing, and more.

What we will learn this week:

- How to write and apply **basic CSS rules to style HTML**
- Understanding and using **selectors** (tag, class, id, attribute, pseudo-class, pseudo-element)
- **CSS specificity** and how the cascade determines which rules win

---

Warm-up (5 min)

Open last week's `profile.html` (or `week-02` page).

1. Add three `<p>` paragraphs in the page (a short intro, a hobby, and a goal for this course)
2. Add `<link rel="stylesheet" href="styles.css" />` in `<head>`
3. Create `styles.css` next to the HTML file
4. Change the paragraph font in CSS, for example:

```
p { font-family: Georgia, "Times New Roman", serif; font-size: 1.125rem; li
ne-height: 1.6; }
```

1. Serve with Live Server — confirm all three paragraphs changed font

If the font does not change, fix the path before anything else today.

CSS Syntax and How to Attach It

A CSS rule has a selector and a declaration block.

Example

```
selector { property: value; } h1 { color: navy; font-size: 2rem; }
```

Example — Three Ways to Attach CSS

```
<!-- 1. Inline. Highest specificity, unmaintainable. Avoid. --> <p style="c
olor: red;">Text</p> <!-- 2. Internal. Fine for a single-file demo. --> <st
yle> p { color: red; } </style> <!-- 3. External. What you use for real wor
k. --> <link rel="stylesheet" href="styles.css"/>
```

Example Explained

Way	Where it lives	Use it?
Inline	`style="…"` on tag	Almost never
Internal	`<style>` in `<head>`	Quick demos only
External	`.css` file + `<link>`	**Yes — professional default**

> **Note:** Use an external stylesheet for all real work this course.

Class activity A — external stylesheet only (5 min)

Paste this into a new file `inline-start.html` (or into your profile page):

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="UTF-8"/> <meta name
="viewport" content="width=device-width, initial-scale=1.0"/> <title>Inline
to External</title> </head> <body style="background: #f4f1ea; font-family:
Georgia, serif;"> <h1 style="color: navy; font-size: 2rem;">My Profile</h1>
<p style="color: #333; font-size: 1.125rem; line-height: 1.6;"> I am learni
ng web engineering this semester. </p> <p style="color: #333; font-size: 1.
125rem; line-height: 1.6;"> I like coffee, walking, and building small webs
ites. </p> </body> </html>
```

Then:

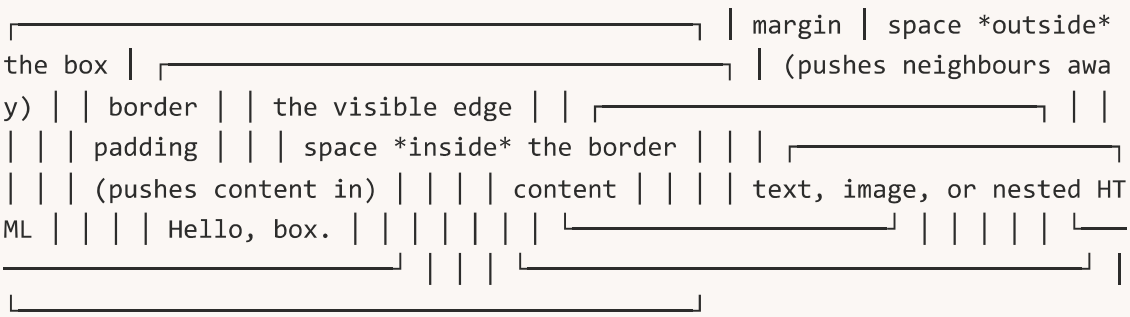
1. Create `styles.css` next to the HTML file
2. Add `<link rel="stylesheet" href="styles.css" />` in `<head>`
3. Move every inline `style="…"` into CSS rules (`body` , `h1` , `p`)
4. Delete all `style="…"` attributes from the HTML

Done when: View Source shows **zero** inline styles, and the page looks the same as before.

The Box Model

Every HTML element is a **box**. CSS does not style "the words" — it styles the box around them, in four layers:

content → padding → border → margin



Example

A small card so you can see each layer:

```
<article class="card"> <h2>Hello, box.</h2> <p>This text is the content. Padding holds it off the border. Margin holds the card off the page edge.</p> </article>
```

```
.card { width: 300px; padding: 20px; /* inside: content ↔ border */ border: 2px solid #333; /* the edge */ margin: 16px; /* outside: this box ↔ neighbours */ background: #e3f2fd; }
```

In the browser you should see: blue card, 20px of blue space around the text, a 2px dark border, then 16px of page showing around the card.

Example Explained

Layer	CSS	What it does
Content	`width`, `height`	Size of the inner area (text / image)
Padding	`padding`	Space between content and border
Border	`border`	The outline
Margin	`margin`	Space between this box and the next

Shorthand order (clockwise: top → right → bottom → left):

```
margin: 10px; /* all four */ margin: 10px 20px; /* vertical | horizontal */
margin: 10px 20px 30px; /* top | horizontal | bottom */ margin: 10px 20px 3
0px 40px; /* top right bottom left */ margin-inline: auto; /* centres a fix
ed-width box */
```

The same four patterns work for `padding`.

> **Note:** Vertical sibling margins merge into the larger value (margin collapse). Horizontal margins never collapse.

Class activity B — see the four layers (10 min)

Paste this starter. Two boxes on a grey page make each layer easy to see.

```
<body> <div class="box">Box 1 — this text is the content.</div> <div class
="box">Box 2 — watch the gap between us.</div> </body>
```

```
body { background: #e8e8e8; } .box { width: 280px; padding: 20px; border: 2 px solid navy; margin: 16px; background: #e3f2fd; }
```

Change **one line at a time**. Save, then look at the page before the next step.

1. Change `padding` to `40px` — the text sits further from the navy edge. That space is **padding** (still blue).
2. Change `border` to `8px solid navy` — the edge gets thicker. That is the **border**.
3. Change `margin` to `40px` — the grey gap between the two boxes grows. That grey is **margin**.
4. press `F12`, click a box, and look at the coloured diagram at the bottom. The numbers should match your CSS.

Done when: you can point at content, padding, border, and margin on the page.

Selectors and Combinators

Selectors pick which elements a rule styles.

Example

```
* { } /* everything */ p { } /* all <p> */ .card { } /* class="card" */ #he ader { } /* id="header" */ [type="email"] { } /* attribute */ a:hover { } / * pseudo-class */ p::first-line { } /* pseudo-element */ .card p { } /* des cendant — any depth */ .card > p { } /* direct child only */ a:focus-visibl e { } /* keyboard focus */ button:disabled { } input:invalid { } li:nth-chi ld(odd) { } .quote::before { content: "\201C"; font-size: 3rem; } .require d::after { content: " *"; color: crimson; }
```

Example Explained

Selector	Matches	Specificity
`*`	Everything	0,0,0
`p`	All `<p>`	0,0,1
`.card`	`class="card"`	0,1,0
`#header`	`id="header"`	1,0,0
`[type="email"]`	Attribute	0,1,0
`a:hover`	Pseudo-class	0,1,0
`p::first-line`	Pseudo-element	0,0,1

Class activity C — selector scavenger hunt (15 min)

Paste this HTML. Do **not** change it. Your job is only the CSS.

```
<body> <h1>Selector hunt</h1> <p>This is a normal paragraph.</p> <p class
="note">This paragraph has a class.</p> <p id="alert">This paragraph has an
id.</p> <ul> <li>HTML</li> <li>CSS</li> <li>Git</li> <li>JavaScript</li> </
ul> <nav> <a href="#skills">Skills</a> <a href="#about">About</a> </nav> <i
nput type="email" placeholder="email input"/> <input type="text" placeholde
r="text input"/> </body>
```

Link a `styles.css`. Apply **all seven rules** one by one. After each one, screenshot the page (for hover: screenshot **while** the mouse is on a link).

```
/* 1. Tag */ h1 { color: navy; font-size: 2rem; } /* 2. Class */ .note { co
lor: teal; font-weight: bold; } /* 3. ID */ #alert { background: #fff3cd; p
adding: 8px; } /* 4. Attribute */ input[type="email"] { border: 2px solid t
eal; padding: 8px; } /* 5. Hover */ a:hover { color: crimson; text-decorati
on: underline; } /* 6. Odd items */ li:nth-child(odd) { background: #e3f2f
d; } /* 7. Even items */ li:nth-child(even) { background: #fff3e0; }
```

What you should see:

1. **Tag** (`h1`) — only the heading is navy and larger.
2. **Class** (`.note`) — only the class paragraph is teal and bold. The other paragraphs do not change.
3. **ID** (`#alert`) — only the id paragraph has a yellow background.
4. **Attribute** (`input[type="email"]`) — the email box has a teal border. The text box does not.
5. **Hover** (`a:hover`) — Skills / About turn crimson when you point at them.
6. **Odd** (`li:nth-child(odd)`) — **HTML** and **Git** have a blue background (1st and 3rd).
7. **Even** (`li:nth-child(even)`) — **CSS** and **JavaScript** have an orange background (2nd and 4th).

Done when: all seven effects show, and you have **seven screenshots** pasted in assignment word file as below

ACTIVITY C

1. Tag (`h1`)

(paste screenshot)

2. Class (`.note`)

(paste screenshot)

3. ID (`#alert`)

(paste screenshot)

4. Attribute (`input[type="email"]`)

(paste screenshot)

5. Hover (`a:hover`)

(paste screenshot while the pointer is on a link)

6. Odd (`li:nth-child(odd)`)

(paste screenshot)

7. Even (`li:nth-child(even)`)

(paste screenshot)

Cascade and Specificity

When two rules conflict, the browser picks a winner.

Example — HTML + CSS (cascade + specificity)

Goal: see which `color` wins when multiple rules match the same element.

HTML

```
<main id="main"> <p class="intro">I am the intro paragraph.</p> </main>
```

CSS (put these in this order)

```
/* 1) Element selector: 0,0,1 */ p { color: green; } /* 2) Class selector: 0,1,0 → beats p */ .intro { color: blue; } /* 3) Element + class: 0,1,1 → beats .intro */ p.intro { color: purple; } /* 4) ID + element: 1,0,1 → beats everything above */ #main p { color: navy; }
```

What you should observe

- The paragraph becomes **navy** because `#main p` has the highest specificity.
- If you **comment out** `#main p`, then the paragraph becomes **purple** (because `p.intro` beats `.intro`).
- If you keep all selectors the same specificity and only change **order**, the **last rule wins** (that's the "cascade / source order" rule).

Bonus: inline style (highest, avoid)

```
<p class="intro" style="color: crimson;">Inline usually wins.</p>
```

Inline styles beat normal stylesheet selectors, which is why we avoid them for maintainability.

Example Explained

The browser uses this order:

1. **Importance** (`!important` beats normal — avoid it)
2. **Specificity** (inline > id > class > element)
3. **Source order** (last wins)

Practical rule: **style with classes.** Avoid IDs in CSS. Flat specificity stays predictable.

> **Warning:** Reaching for `!important` usually means your selectors are fighting. Fix the selector instead.

Class activity D — “who wins and why”(8 min)

Part 1) Implement the “specificity duel”

1. In your HTML, create this structure (or match it closely):

```
<main id="main"> <p class="intro">I am the intro paragraph.</p> </main
>
```

2. In your `styles.css`, add these rules **in this exact order**:

```
/* 1) Element selector: 0,0,1 */ p { color: green; } /* 2) Class selec
tor: 0,1,0 */ .intro { color: blue; } /* 3) Element + class: 0,1,1 */
p.intro { color: purple; } /* 4) ID + element: 1,0,1 */ #main p { colo
r: navy; }
```

Part 2) Screenshots to submit (5 screenshots)

Take screenshots showing both the page and DevTools → **Elements** → **Styles** (so we can see the crossed-out rules).

1. **Screenshot #1 (all 4 rules active)**
   - Show the paragraph colour and Styles panel.
2. **Screenshot #2 (comment out `#main p`)**
   - Comment out only this block:

```
/* #main p { color: navy; } */
```

   - Screenshot again.

3. **Screenshot #3 (comment out `p.intro` too)**
    - Now comment out:

```
/* p.intro { color: purple; } */
```

    - Screenshot again.
4. **Screenshot #4 (source order test: same specificity)**
    - Keep only two rules with the **same selector** and change their order, for example: